

实用

- 思想
- 对拍
- pbds
- 双指针
- bitset
- 随机
- 输入输出流
- STL
- 运算优先级

无用

- 高精度

实用

思想

遇到傻逼题先想必要条件缩小题目限制

当数据缩小为 50 以内可以考虑折半

遇到实在写不出正确复杂度，可以考虑随机选取，然后跑 T 次，一次的正确概率为 p ，则 T 次错误概率为 $(1 - p)^T$ ，期望 $\frac{1}{p}$ 次找到答案

对拍

```
1 @echo off
2 g++ -std=c++17 -O2 gen.cpp -o gen
3 g++ -std=c++17 -O2 %1.cpp -o %1
4 g++ -std=c++17 -O2 %2.cpp -o %2
5 :loop
6 gen > gen.in
7 %1 < gen.in > %1.out
8 %2 < gen.in > %2.out
9 fc %1.out %2.out
10 if errorlevel 1 exit /b
11 goto loop
```

```
1 #!/bin/bash
2 g++ -std=c++17 -O2 gen.cpp -o gen
3 g++ -std=c++17 -O2 "$1.cpp" -o "$1"
4 g++ -std=c++17 -O2 "$2.cpp" -o "$2"
5
6 while true; do
7     ./gen > gen.in
8     ./"$1" < gen.in > "$1.out"
9     ./"$2" < gen.in > "$2.out"
```

```

10     if diff "$1.out" "$2.out" > /dev/null; then
11         echo "Accepted"
12     else
13         echo "Wrong Answer"
14         exit 1
15     fi
16 done

```

```

1  chmod +x duipai.sh
2  ./duipai.sh program1 program2

```

pbds

```

1  #include <bits/stdc++.h>
2  #include <ext/pb_ds/assoc_container.hpp>
3  #include <ext/pb_ds/priority_queue.hpp>
4  #include <ext/rope>
5  using namespace __gnu_pbds;
6  using namespace __gnu_cxx;
7  using namespace std;
8  #define int long long
9  mt19937_64 rd(time(0));
10 template<class key, class val = null_type, class cmp = less<>>
11 using rb_tree = tree<key, val, cmp, rb_tree_tag,
12 tree_order_statistics_node_update>;
13 template<class T, class ... A>
14 void debug(const T & t, const A & ... a) { cerr << "[" << t, ((cerr << ", " <<
15 a), ...), cerr << "]\n"; }
16
17 void QAQ() {
18
19 }
20
21 signed main() {
22     int t = 1;
23     cin >> t;
24     for (cin.tie(0) -> sync_with_stdio(0), cout << fixed << setprecision(12); t-
25 -; QAQ());
26 }

```

堆： 参数不用 `vector`，`push` 返回 `point_iterator`，配合数组记录元素堆内位置，为 `NULL` 时表示不在堆内

- `push`, `join` 为 $O(1)$ ，`pop`, `modify`, `erase` 均摊 $O(\log n)$ ，最坏 $O(n)$
 1. `modify(point_iterator, const key)` 修改
 2. `erase(point_iterator)` 删除
 3. `join(pairing_heap & other)` 合并后 `other` 清空

平衡树： 以下操作均为 $O(\log n)$

1. `find_by_order(int k)`：需保证 $k \in [0, size)$ 返回第 $k + 1$ 小的迭代器，使用时 k 需要提前 -1

2. `order_of_key(key)` : 返回以 `cmp` 比较, 小于 `key` 的个数, `key` 可以不用在树中, 排名需额外 +1
3. `join(rb_tree & other)` : 需保证键的值域不相交, 合并后 `other` 清空
4. `split(const key, rb_tree & other)` : 以 `cmp` 比较, 小于等于 `x` 的属于当前树, 大于的属于 `other`

双指针

使用双指针维护区间信息也可以与其他数据结构比如差分、单调队列、线段树、主席树等等结合使用

bitset

! 空间开不下的时候, 可以考虑根号分治

初始化:

- `bitset<n> a(b)` : 使得 `a` 是 `b` (可以是 `unsigned` 或 `string`) 的副本
- `bitset<n> a(b, pos, n)` : 使得 `a` 是 `b` 从 `pos` (默认为 0) 开始 `n` 个位的副本, `n` 可以没有, 这样就是到结束

函数:

- `count()` : 返回 `true` 的数量
- `none()` : 若所有位都是 `false` 则返回 `true`, 否则返回 `false`
- `all()` : 若所有位都是 `true` 则返回 `true`, 否则返回 `false`
- `set()` : 将整个 `bitset` 设置为 `true`
- `reset()` : 将整个 `bitset` 设置为 `false`
- `flip()` : 翻转每一位
- `to_string()` : 返回转换成的字符串表达
- `to_ulong()` (不建议使用) : 返回转换成的 `unsigned long` 表达
- `to_ullong()` : 返回转换成的 `unsigned long long` 表达
- `_Find_first()` : 返回 `bitset` 第一个 `true` 的下标, 若没有 `true` 则返回 `bitset` 的大小
- `_Find_next(pos)` : 返回 `pos` 后面 (严格大于) 第一个 `true` 的下标, 若没有 `true` 则返回 `bitset` 的大小

动态开 `bitset` :

```
1  template <int len = 1>
2  void QAQ(int n, const int MAXN) {
3      if (len < n) {
4          QAQ<min(len * 2, MAXN)>(n, MAXN);
5          return;
6      }
7      bitset<len + 1> bit;
8  }
```

随机

```
1 mt19937_64 rd(time(0));
2 shuffle(a.begin() + 1, a.end(), rd); //随机打乱数组
```

输入输出流

- `getline(cin, string)` : `cin` 遇到回车结束输入, 但回车还在输入流中, `cin` 并不会清除, 导致 `getline(cin, string)` 读取回车, 结束, 需要在 `cin` 后面加 `cin.get()` 或 `cin.ignore()` 主动删除输入流中的换行符
- 解绑: `cin.tie(0) -> sync_with_stdio(0)` , 解绑后不能与 `scanf, getchar, printf, cin.getline()` 混用
- 精度: 只使用 `setprecision(n)` 为整个数字设置有效位数 (默认为 6) , 搭配 `fixed` 设置成小数部分的有效位数, 搭配 `scientific` 设置科学计数法的有效位数

```
1 cout << fixed; //转化为小数有效位数
2 cout << scientific; //转化为科学计数法的有效位数
3 cout << setprecision(n); //设置有效位数精度为 n
4 cout << defaultfloat; //恢复默认小数的显示方式
```

- 进制: 使用 `hex` 、 `oct` 、 `dec` 来控制输出的进制 (默认为 `dec`) , 或者 `setbase(n)` 设置 `n` 进制, 取值为 8、10 或者 16 , 其余值无输出, 使用 `cout << uppercase` 可以输出进制成大写字母 (默认为 `cout << lowercase`)
- 宽度: `cout << setw(n)` 设置输出宽度 (默认为 0) , 搭配 `setfill()` 可以设置填充什么 (默认为空格) , 同时可以使用 `cout << left` 或 `cout << right` 实现左右对齐 (默认为 `right`)
- 显示符号 (正) : `cout << showpos` (默认使用 `cout << noshowpos`)
- 设置多种属性:
 - `setiosflags(flag)` : 将某个输出格式标志设置为 1 , 对于以上非函数的属性都可以使用 `|` 连接, `cout << setiosflags(ios::fixed | ios::showpos | ios::left | ios::hex | ios::uppercase)`
 - `resetiosflags(flag)` : 将某个输出格式标志设置为 0

STL

容器初始化:

- 通用: 都可以使用 `vector` 等数组的迭代器区间初始化, `erase()` 可以是迭代器区间
- `string` :
 - `string a("114514", pos, n)` : 使得 `a` 是 `string, const char *` 等字符串类型从 `pos` (默认 0) 开始 `n` 个位的副本, `n` 可以没有, 这样就是到结束

容器函数及其特性:

- `string` :

- `c_str()` : 将 `string` 转换为 `const char *`
- `replace(it1, it2, str)` : 将字符串 `[it1, it2)` 区间替换成 `str`
- `substr(pos, n)` : 截取从 `pos` 索引开始的 `n` 个字符串
- `set, map, priority_queue` 重载比较函数: 记得加 `const` 防止一些函数使用不了

```
1 set<int, function<bool(int, int)>> s([&](const auto & x, const auto & y) {
2     return x < y;
3 });
```

- `multiset, multimap` 删除: 删除一个元素使用 `extract()` 删除值或迭代器, 或者使用 `erase()` 删除迭代器, 删除全部使用 `erase()` 删除值
 - `multimap` 直接 `extract()` 键值的行为未定义
 - 使用 `extract()` 返回一个 `std::node_type` 结点柄, 找不到返回空结点柄
 - `set.extract()` 得到的结点柄使用 `.value()` 获取值
 - `map.extract()` 得到的结点柄使用 `.key()` 获取键值, 使用 `.mapped` 获取值
- `set` 等容器查询是否存在: 使用 `find() != end()`, `count()` 或 `contains()`, 但 `contains()` 只在 C++20 及以上才有
- `unordered_map` 重载哈希函数:

```
1 unordered_map<pair<string, int>, int, function<size_t(pair<string, int>)>> mp(0,
2     [&](const auto & k) {
3         return hash<string>()(k.first) ^ hash<int>()(k.second);
4     });
5 gp_hash_table<pair<string, int>, int, function<size_t(pair<string, int>)>> mp([&]
6     (const auto & k) {
7         return hash<string>()(k.first) ^ hash<int>()(k.second);
8     });
```

- `deque` 排序: 可以使用 `sort` 直接进行排序
- `vector<bool>` 注意事项: 无法使用引用和指针

STL函数:

- `lower_bound, upper_bound` 重载比较函数: 返回条件都是 `x < y`
 - `lower_bound` 查询匿名函数第一个为 `false`, 传的参数第一个为 `x` 为 数组元素, 第二个为 `y` 为 查找元素
 - `upper_bound` 查询匿名函数第一个为 `true` 传的参数第一个为 `x` 为 查找元素, 第二个为 `y` 为 比较函数

```
1 lower_bound(a.begin(), a.end(), t, [&](auto & x, auto & y) {
2     return x < y;
3 }); //x 为数组元素, y 为查找元素
4 upper_bound(a.begin(), a.end(), t, [&](auto & x, auto & y) {
5     return x < y;
6 }); //x 为查找元素, y 为数组元素
```

- `accumulate` 求和:

```

1 accumulate(a.begin(), a.end(), 0ll, [&](auto now, auto & x) {
2     return now + x;
3 }); //now 为累加的值, x 为数组元素

```

- **fill(a.begin(), a.end(), x) 填充**: 将区间赋值为 x
- **is_sorted(a.begin(), a.end()) 判断有序**: 判断是否有序, 返回 bool
- **iota(a.begin(), a.end(), 0) 递增赋值**: 让序列递增赋值
- **copy(a.begin(), a.end(), b.begin() + pos) 拷贝赋值**: 让 a 区间赋值给 b 从 pos 开始范围

• 大小有关函数:

- **minmax({a, b})**: 返回 pair, first 最小, second 最大, !! 使用时一定要 minmax({a, b})
- **minmax_element**: 返回 pair 迭代器

```

1 minmax_element(a.begin(), a.end(), [&](auto & x, auto & y) {
2     return x < y;
3 })

```

- **nth_element**: 寻找序列第 k 小, 使得 a.begin() + k - 1 之前的元素都小于等于它, 之后的元素都大于等于它 (是赋值的)

```

1 nth_element(a.begin(), a.begin() + k - 1, a.end(), [&](auto & x, auto & y) {
2     return x < y;
3 });

```

◦ 排列:

- **next_permutation(a.begin(), a.end())**: 如果是最大排列, 重排为最小排列, 返回 false, 否则求出下一个序列后, 返回 true (按字典序)
- **prev_permutation(a.begin(), a.end())**: 如果是最小排列, 重排为最大排列, 返回 false, 否则求出前一个排列后, 返回 true

• 转化函数:

◦ 字符串转 signed, unsigned:

- **strtoi(str, nullptr, 10), strtoll(str, NULL, 0), strtoui(...), strtoull(...)**: 将 const char * 转化成 signed, unsigned (必须有三个参数), 第三个参数为进制 (0到36之间), 如果 base 为 0, 那么 strtoll 会根据 str 的前缀来判断基数: 如果 str 以 "0x" 或 "0X" 开始, 将按照十六进制处理; 如果 str 以 "0" 开始, 将按照八进制处理; 其他情况按照十进制处理。

- **stoi(str), stoll(str, nullptr, 10), stoui(str, NULL, 2), stoull(...)**: 将 string 转化为 signed, unsigned (可以只有一个或者同时有三个参数)

◦ 字符串转浮点:

- **strtod(str, nullptr), strtod(str, NULL), strtold(...)**: 将 const char * 转化为 double, float, long double (必须有俩个参数)
- **stod(str), stof(str), stold(...)**: 将 string 转化为 double, float, long double

◦ 转字符串:

- **to_string()**: 大多数正常类型转字符串

◦ 大小写转化:

- `tolower(char), toupper(char)` :
- 使用 `transform` 算法配合: 有 4 个参数, 前 2 个指定要转换的容器的起止范围, 第 3 个参数是结果存放容器的起始位置, 第 4 个参数是一元运算

```
1 transform(s.begin(), s.end(), s.begin(), [&](auto & x) {
2     return ::tolower(c);
3 }); //转换小写, ::tolower 表示全局, 无 :: 表示优先作用域
4 transform(s.begin(), s.end(), s.begin(), ::toupper); //转换大写
```

- `move` 移动函数: 将左值强转成右值, 防止深拷贝, 相当于改变作用域, 使用后无法再次访问
- `decltype` 类型推导: 用于推导类型, 可以推导出引用, 但是不能推导出数组
- `tie` 解包: 用于解包 `pair` 或者 `tuple`, 可以用 `ignore` 忽略某个值

运算优先级

高优先级:

1. 成员访问和函数调用: `a.b`, `a->b`, `a[b]`, `a()`
2. 后置自增自减: `a++`, `a--`
3. 前置自增自减和单目运算: `++a`, `--a`, `!a`, `~a`
4. 类型转换: `(type)a`
5. 指针运算: `*a`, `&a`
6. `sizeof` 运算: `sizeof a`, `sizeof(type)`
7. `new` 和 `delete`: `new`, `new[]`, `delete`, `delete[]`

中等优先级:

8. 乘除和取模: `a * b`, `a / b`, `a % b`
9. 加减: `a + b`, `a - b`
10. 位移: `a << b`, `a >> b`
11. 关系运算: `a < b`, `a <= b`, `a > b`, `a >= b`
12. 相等运算: `a == b`, `a != b`
13. 按位与: `a & b`
14. 按位异或: `a ^ b`
15. 按位或: `a | b`
16. 逻辑与: `a && b`
17. 逻辑或: `a || b`

低优先级:

18. 条件运算: `a ? b : c`
19. 赋值和复合赋值: `a = b`, `a += b`, `a -= b`, `a *= b`, `a /= b`, `a %= b`, `a <= b`, `a >= b`, `a &= b`, `a ^= b`, `a |= b`
20. 逗号运算: `a, b`

无用

高精度

```
1  #include<iostream>
2  #include<cstring>
3  #include<cstdio>
4  #include<algorithm>
5  using namespace std;
6  const int L=2015;//L为长度大小
7  string add(string a,string b)//只限两个非负整数相加
8  {
9      string ans;
10     int na[L]={0},nb[L]={0};
11     int la=a.size(),lb=b.size();
12     for(int i=0;i<la;i++) na[la-1-i]=a[i]-'0';
13     for(int i=0;i<lb;i++) nb[lb-1-i]=b[i]-'0';
14     int lmax=la>lb?la:lb;
15     for(int i=0;i<lmax;i++) na[i]+=nb[i],na[i+1]+=na[i]/10,na[i]%=10;
16     if(na[lmax]) lmax++;
17     for(int i=lmax-1;i>=0;i--) ans+=na[i]+'0';
18     return ans;
19 }
20 string sub(string a,string b)//只限大的非负整数减小的非负整数
21 {
22     string ans;
23     int na[L]={0},nb[L]={0};
24     int la=a.size(),lb=b.size();
25     for(int i=0;i<la;i++) na[la-1-i]=a[i]-'0';
26     for(int i=0;i<lb;i++) nb[lb-1-i]=b[i]-'0';
27     int lmax=la>lb?la:lb;
28     for(int i=0;i<lmax;i++)
29     {
30         na[i]-=nb[i];
31         if(na[i]<0) na[i]+=10,na[i+1]--;
32     }
33     while(!na[--lmax]&& lmax>0) ;lmax++;
34     for(int i=lmax-1;i>=0;i--) ans+=na[i]+'0';
35     return ans;
36 }
37 string mul(string a,string b)//高精度乘法a,b,均为非负整数
38 {
39     string s;
40     int na[L],nb[L],nc[L],La=a.size(),Lb=b.size();//na存储被乘数,nb存储乘数,nc存储
    积
41     fill(na,na+L,0);fill(nb,nb+L,0);fill(nc,nc+L,0);//将na,nb,nc都置为0
42     for(int i=La-1;i>=0;i--) na[La-i]=a[i]-'0';//将字符串表示的大整形数转成i整形数组表
    示的大整形数
43     for(int i=Lb-1;i>=0;i--) nb[Lb-i]=b[i]-'0';
44     for(int i=1;i<=La;i++)
45         for(int j=1;j<=Lb;j++)
46             nc[i+j-1]+=na[i]*nb[j];//a的第i位乘以b的第j位为积的第i+j-1位(先不考虑进位)
47     for(int i=1;i<=La+Lb;i++)
```

```

48         nc[i+1]+=nc[i]/10,nc[i]%10;//统一处理进位
49         if(nc[La+Lb]) s+=nc[La+Lb]+'0';//判断第i+j位上的数字是不是0
50         for(int i=La+Lb-1;i>=1;i--)
51             s+=nc[i]+'0';//将整形数组转成字符串
52         return s;
53     }
54     int sub(int *a,int *b,int La,int Lb)
55     {
56         if(La<Lb) return -1;//如果a小于b, 则返回-1
57         if(La==Lb)
58         {
59             for(int i=La-1;i>=0;i--)
60                 if(a[i]>b[i]) break;
61                 else if(a[i]<b[i]) return -1;//如果a小于b, 则返回-1
62         }
63     }
64     for(int i=0;i<La;i++)//高精度减法
65     {
66         a[i]-=b[i];
67         if(a[i]<0) a[i]+=10,a[i+1]--;
68     }
69     for(int i=La-1;i>=0;i--)
70         if(a[i]) return i+1;//返回差的位数
71     return 0;//返回差的位数
72 }
73 }
74 string div(string n1,string n2,int nn)//n1,n2是字符串表示的被除数, 除数,nn是选择返回商
    还是余数
75 {
76     string s,v;//s存商,v存余数
77     int a[L],b[L],r[L],La=n1.size(),Lb=n2.size(),i,tp=La;//a, b是整形数组表示被除
    数, 除数, tp保存被除数的长度
78     fill(a,a+L,0);fill(b,b+L,0);fill(r,r+L,0);//数组元素都置为0
79     for(i=La-1;i>=0;i--) a[La-1-i]=n1[i]-'0';
80     for(i=Lb-1;i>=0;i--) b[Lb-1-i]=n2[i]-'0';
81     if(La<Lb || (La==Lb && n1<n2)) {
82         //cout<<0<<endl;
83         return n1;}//如果a<b, 则商为0, 余数为被除数
84     int t=La-Lb;//除被数和除数的位数之差
85     for(int i=La-1;i>=0;i--)//将除数扩大10^t倍
86         if(i>=t) b[i]=b[i-t];
87         else b[i]=0;
88     Lb=La;
89     for(int j=0;j<=t;j++)
90     {
91         int temp;
92         while((temp=sub(a,b+j,La,Lb-j))>=0)//如果被除数比除数大继续减
93         {
94             La=temp;
95             r[t-j]++;
96         }
97     }
98     for(i=0;i<L-10;i++) r[i+1]+=r[i]/10,r[i]%10;//统一处理进位
99     while(!r[i]) i--;//将整形数组表示的商转化成字符串表示的
100    while(i>=0) s+=r[i--]+'0';
101    //cout<<s<<endl;

```

```

102     i=tp;
103     while(!a[i]) i--; //将整形数组表示的余数转化成字符串表示的
104     while(i>=0) v+=a[i--]+'0';
105     if(v.empty()) v="0";
106     //cout<<v<<endl;
107     if(nn==1) return s;
108     if(nn==2) return v;
109 }
110 bool cmp(string a,string b)
111 {
112     if(a.size()<b.size()) return 1; //a小于等于b返回真
113     if(a.size()==b.size()&&a<=b) return 1;
114     return 0;
115 }
116 string BigInterSqrt(string n)
117 {
118     string l="1",r=n,mid,ans;
119     while(cmp(l,r))
120     {
121         mid=div(add(l,r),"2",1);
122         if(cmp(mul(mid,mid),n)) ans=mid,l=add(mid,"1");
123         else r=sub(mid,"1");
124     }
125     return ans;
126 }
127 string DeletePreZero(string s)
128 {
129     int i;
130     for(i=0;i<s.size();i++)
131         if(s[i]!='0') break;
132     return s.substr(i);
133 }
134 int main()
135 {
136     //freopen("in.txt","r",stdin);
137     // freopen("out.txt","w",stdout);
138     string n;
139     int t;
140     cin>>t;
141     while(t--)
142     {
143         cin>>n;
144         n=DeletePreZero(n);
145         cout<<BigInterSqrt(n)<<endl;
146         //cout<<BigInterSqrt(n).size()<<endl;
147     }
148     return 0;
149 }

```

